

OGGI Wholesale v2 — Technical Overview & Handover

Prepared: 15 August 2026 **Audience:** CTO / engineering lead, ahead of QA and independent testing **Status:** Live, database-connected, core buyer order flow verified end-to-end on real data (first real order placed 15 Aug 2026). Not yet security-hardened for public production — see §9.

This document describes **what exists and how it is built**, not what to build. It is written to let an engineer who has never seen this codebase orient themselves, run it, deploy it, and understand its architecture and open risks. Everything below was verified against the live system and the actual source on 15 August 2026, not assumed.

1. What the system is

OGGI Wholesale v2 is a **multi-tenant B2B wholesale ordering platform** — a single web app that serves several distinct roles, each with its own screens:

- **Buyers** browse a wholesaler's catalogue and place orders (open stock or fixed bundles — see selling models below).
- **Wholesalers** manage their own products, pricing, stock, packs, clients, and staff.
- **Salespeople** place orders on behalf of clients they manage.
- **Warehouse** staff pick and fulfil orders (barcode/scan support).
- **Owner (OGGI)** oversees every wholesaler in the system, issues access, and sees cross-tenant aggregates.

What makes it *wholesale* rather than retail — and what a tester should focus on — is the ordering rules layer: per-SKU and per-order **minimum order quantities**, colour × size **variant matrices**, and four **selling models**:

1. **Open stock** — pick any quantity of any size.
2. **Prepack / fixed carton** — a fixed carton per colour.
3. **Ratio pack** — the wholesaler's size-mix curve, sold as a unit per colour.
4. **Full series** — every colour and size together as one orderable unit.

All four are enforced server-side (a buyer cannot build an order the server would reject). This was the single hardest part of the build and the last to be verified: as of 15 Aug 2026 a full-series pack was ordered end-to-end through the live UI and recorded correctly (16 SKUs collapsed into one order line).

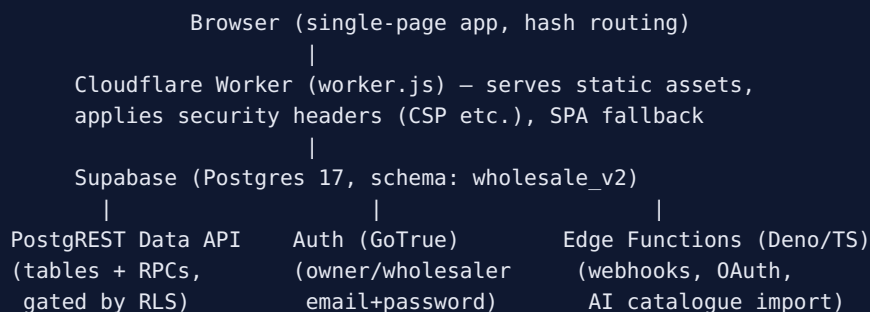
Beyond ordering, the platform includes: multi-location inventory with soft reservations, inventory intelligence (reorder points, dead-stock/ABC analysis, landed cost, GMROI), kit assembly, per-client negotiated pricing, CSV import/export, AI catalogue import from a photo, and outbound integrations (Shopify / WooCommerce / WhatsApp). Their build state is inventoried in `FEATURE-MANIFEST.md` (see §8).

2. Live URLs and infrastructure

Thing	Value
Live app	https://oggi-wholesale-v2.oggi-teamz.workers.dev
Hosting	Cloudflare Workers (static-assets binding, not Pages)
Database / backend	Supabase project <code>oggi-wholesale</code> (ref <code>olaipgdckbgjediddloj</code>), region <code>eu-central-1</code> , status <code>ACTIVE_HEALTHY</code> , Postgres 17
Source repository	https://github.com/oggiteamz-dot/oggi-wholesale-v2 (public)
Deploy pipeline	GitHub <code>main</code> → Cloudflare Workers Builds → auto-deploy on push (no manual deploy step)
PWA	Installable, offline-capable (service worker + manifest + icons). Android TWA build documented but not yet produced — see docs/PWA-AND-TWA.md .

v1 is a separate, still-live app (`oggi-wholesale.oggi-teamz.workers.dev`) sharing the same Supabase project but the `public` schema. v2 lives in its own `wholesale_v2` schema. They share one Supabase Auth user pool. v2 does not touch v1.

3. Architecture at a glance



Three design decisions define this system. An engineer must understand all three:

3a. Dedicated schema (`wholesale_v2`)

Every v2 table and function lives in the `wholesale_v2` Postgres schema, isolated from v1's `public` schema. Because PostgREST only exposes whitelisted schemas, **`wholesale_v2` must be listed in the Supabase dashboard under Project → Data API → Settings → Exposed schemas**. If it is not, every data call fails with `Invalid schema: wholesale_v2` and the app appears completely broken. (This exact misconfiguration blocked the app until it was fixed on 15 Aug 2026. It is now set correctly: `public, graphql_public, wholesale_v2`.) This is the first thing to check if the app ever goes dark after an environment change.

3b. Two separate auth systems (critical)

This is the most important thing to understand about the codebase, and the source of the one systemic bug found during testing.

- **Owner and Wholesaler** authenticate through **real Supabase Auth** (email + password → a JWT). Their identity in the database is `auth.uid()`, and a row in `wholesale_v2.v2_user_profiles` maps that auth user to a `wid` and role. RLS helper `v2_my_wid()` reads `wid` from `v2_user_profiles WHERE id = auth.uid()`.
- **Buyer and Salesperson** authenticate through a **custom RPC** (`v2_buyer_login` / `v2_sales_login`), which checks a bcrypt-hashed password in `wholesale_v2.v2_portal_accounts` and returns a client-side session. **These users have NO `auth.uid()`** — for them `auth.uid()` is NULL and `v2_my_wid()` returns NULL.

Consequence every developer must internalise: any table a buyer or salesperson needs to *read* must grant the no-JWT case explicitly, using an `(auth.uid() IS NULL)` clause in its RLS `SELECT` policy — exactly as `v2_products` and `v2_product_variants` do. A table that omits this is invisible to buyers **with no error**. This is precisely how the "packs unorderable" bug hid until a human walked the buyer flow: `v2_pack_definitions` was missing that clause, so every series/prepack/ratio product silently showed "no bundles set up." Fixed in migration `v2_pack_definitions_buyer_read_parity` (repo 033). See §9 for the standing audit rule this created.

3c. Server is the source of truth; the client is a mirror

Writes that touch money or stock never trust the client. Order submission, stock reservation/decrement, pricing, and MOQ are all enforced by **SECURITY DEFINER RPCs** and RLS, not by the front-end. The server recomputes price itself (override → tier → base) and ignores any `unit_price` or `buyer_label` the client sends; it derives them from the authenticated account. The front-end data modules (`js/data/*.js`) are convenience mirrors for UX, clearly commented as such. Stock changes go through an append-only ledger with atomic RPCs, never direct table writes.

4. Database

Verified live on 15 Aug 2026 (schema `wholesale_v2`):

Metric	Value
Base tables	35
Functions (RPCs)	32
Tables with Row-Level Security enabled	35 of 35 (100%)
Supabase Auth users	6
Wholesalers (test data)	4
Products (test data)	9
Orders	1 (the verification order)

Key tables: `v2_wholesalers`, `v2_products`, `v2_product_variants` (colour × size in `extra_attrs`), `v2_inventory_balances`, `v2_stock_reservations`, `v2_pack_definitions` + `v2_pack_components`, `v2_orders` + `v2_order_items`, `v2_portal_accounts` (buyer/sales/wholesaler-staff logins), `v2_user_profiles` (owner/wholesaler auth-user mapping), `v2_invites`, `v2_clients`, integration/secret tables.

The database is built entirely through **numbered SQL migrations** in `supabase/migrations/`, and this is where an engineer should read the schema's history — each migration's header explains *why* it exists, not just what it does

(the audit rated migration commenting "well above typical handover quality"). The migration discipline is: additive-only, atomic-RPC writes, values sourced from v1's real data rather than guessed.

Repo/live migration drift — action required before relying on the repo to rebuild the DB. See §9, item 1. Four migrations are live but were never saved as files. A `supabase db reset` from the current repo would *not* reproduce the live database.

5. Edge Functions (server-side, Deno/TypeScript)

Eight functions are deployed and ACTIVE on the live project. `verify_jwt` status verified live:

Function	In repo?	verify_jwt (live)	Purpose
<code>extract-catalog-from-image</code>	Yes	ON	AI catalogue import from a product photo/PDF (Anthropic)
<code>integration-dispatch</code>	Yes	off	Outbound integration event dispatcher
<code>oauth-connect</code>	Yes	off	OAuth connect flow for Shopify/Woo
<code>shopify-order-webhook</code>	Yes	off	Inbound Shopify order → stock decrement
<code>woocommerce-order-webhook</code>	Yes	off	Inbound WooCommerce order → stock decrement
<code>whatsapp-webhook</code>	Yes	off	Inbound WhatsApp events (log-only)
<code>manage-wholesaler-login</code>	No	off	(Live only — not in repo folder)
<code>minds-bank</code>	No	off	(Unrelated project — different <code>minds</code> schema)

Note the cross-checks: `extract-catalog-from-image` is **more locked down live** (JWT required) than its source comment implies, and `manage-wholesaler-login` exists live but is **not in the repo** — another live/repo drift item to reconcile. The webhook security posture is discussed in §9.

6. Codebase map — how to navigate it

The front-end is **vanilla JavaScript ES modules** (no framework, no build step) — files are served as-is. This keeps it simple and dependency-light; the trade-off is manual DOM rendering. Conventions are consistent across the whole codebase, so learning one file teaches you the rest: every file has a purpose header, every view exports a `register*Routes(router)` function, sections are fenced with `// ----- Name -----` banners, and cross-references cite the governing migration by number.

```

wholesale-v2/
├── index.html          App shell + baseline CSP
├── worker.js           Cloudflare Worker: serves assets + security headers (authoritative)
├── sw.js              Service worker (offline / PWA, stale-while-revalidate)
├── manifest.json       PWA manifest
├── wrangler.toml       Cloudflare deploy config (+ .assetsignore keeps supabase/ private)
├── _headers            Cloudflare header fallback layer
├── js/
│   ├── app.js          Boots the app, mounts the shell, wires routing
│   ├── lib/
│   │   ├── dev-auth.js  ★ The two-auth-system core – READ THIS FIRST
│   │   ├── supabase-client.js ★ Supabase client + schema config
│   │   ├── router.js    Hash router
│   │   ├── nav-config.js Per-role navigation
│   │   ├── utils.js     esc()/money()/pageHeader() – HTML-escaping lives here
│   │   ├── animations/  UX micro-interactions (well-isolated, teardown-safe)
│   │   └── vendor/       Vendored supabase-js (3rd-party, do not edit)
│   ├── data/           One module per domain (cart, catalog, pricing, orders,
│   │                   prepacks, inventory-*, kits, clients, integrations, ...).
│   │                   These are UX mirrors; the server is authoritative.
│   ├── components/     Reusable UI (product-card, topbar, sidenav, toast, ...)
│   └── views/           One file per role/screen group:
│       ├── login.js     All role login forms
│       ├── buyer.js    Buyer catalogue, cart, orders
│       ├── wholesaler.js ▲ 68KB monolith – 11 screens (see §9 item 5)
│       ├── owner.js     Owner dashboard, wholesaler admin, invites
│       ├── salesperson.js Order-on-behalf-of-client
│       ├── mobile-ops.js Warehouse pick/scan
│       ├── integrations.js Integration setup
│       └── import-catalog.js AI + CSV catalogue import
├── supabase/
│   ├── migrations/     Numbered SQL – the schema's real history (read headers)
│   └── functions/       Edge functions (Deno/TS)
├── checks/             Executable verification (see §7)
├── docs/               Batch-by-batch deploy records + PWA/TWA notes
└── FEATURE-MANIFEST.md Feature inventory with per-feature verification status

```

Suggested reading order for a new engineer: `lib/dev-auth.js` and `lib/supabase-client.js` (auth model) → a migration or two (e.g. `004_v2_orders_core`, `029_v2_series_selling_model`) → `js/data/cart.js` (reservation + submit model) → `js/components/product-card.js` and `js/views/buyer.js` (the buyer path) → `FEATURE-MANIFEST.md` for the full inventory.

7. Verification / checks

`checks/` contains **executable, negative-tested** verification — the project's stated rule is "a check is not finished until it has been proven to go red." The audit confirmed these assert real behaviour, not trivial passes:

- `check_data_invariants.sql` — order-line shape (would catch the historical colour×size loss), variant axis integrity, stock sanity (no negatives, no over-reservation), pack integrity, selling-model data-vs-enforcement, $MOQ \geq 1$.
- `check_tenant_isolation.sql` — the production security gate: anon has no table-wide read, private columns (cost, reorder points) are hidden, no `using(true)` read policies, RLS on for all `v2_*` tables.

- `check_service_worker.mjs` — drives real fetch events against `sw.js` (offline fallback, cache poisoning prevention, Supabase never cached).

`FEATURE-MANIFEST.md` ties each shipped feature to the file it lives in and the check that proves it still works, so a feature cannot silently disappear across a rewrite.

Note: `checks/README.md` references four check files that are not on disk (`check_pack_moq.sh` , `check_escaping.mjs` , `fixture.sql` , `seed.sql`) — same "documented but not saved" pattern as the missing migrations. See §9.

8. Feature inventory

The authoritative, per-feature build status lives in `FEATURE-MANIFEST.md` in the repo. As of 15 Aug 2026 it lists 32 tracked features: 17 enforced-and-proven, 15 present-but-not-yet-gated, 0 not-built, 0 lost. All four selling models are enforced end-to-end, now verified by a real order. Read that file for the full breakdown; it is kept current as features are proven.

9. Known open items and risks (read before production)

These are stated plainly so nothing is discovered the hard way. None block QA/testing; several block *public* production.

1. Repo cannot rebuild the database from scratch (highest priority). Four migrations are applied to the live DB but were never saved as files in the repo. From the live migration history they are: - `v2_pack_line_validation` (2026-08-15) — the missing repo `028` - `v2_ratio_and_prepack_selling_models` (2026-08-15) — missing `030` - `v2_tenant_isolation_cost_and_reads` (2026-08-15) — missing `031` - `v2_tenant_isolation_cost_column_grants` (2026-08-15) — missing `032`

These are load-bearing (pack-line validation, ratio/prepack enforcement, and the cost-column tenant isolation). Until they are exported from the live project and committed, a fresh rebuild would be missing real security and enforcement logic. **Recommended: dump these from the live project and commit them.** (This can be done from the current session.)

2. Webhook/edge-function security is in a documented "dev-mode" posture. For the order webhooks (Shopify/WooCommerce), HMAC signature verification is **opt-in** — if no `webhook_secret` is saved for a wholesaler, the request is processed and **real stock is decremented** with `verified=false`. The WhatsApp POST path has no signature check (log-only, lower impact). `verify_jwt` is off on the webhook/dispatch/oauth functions. OAuth `state` is unsigned, and HMAC comparison is not constant-time. All of this is honestly documented in the code as pre-hardening items, but a security reviewer must sign these off — and webhook secrets must be enforced — before the integration endpoints are exposed publicly. (Good news: **no secrets are hardcoded** — service-role keys, the Anthropic key, and OAuth secrets are all read from environment/vault; only the public publishable key and project URL appear in source, which is expected.)

3. Ships a "dev" badge and a mislabeled button. `topbar.js` hardcodes a `v2 · dev` environment tag, and the "Switch role" button actually performs a full sign-out (leftover dev-mode wording from before real auth). Cosmetic, but should be corrected before it reaches real users.

4. Two small code-level defects found in the audit (neither blocks testing): - `js/data/prepacks.js` — the `suggestPackRatio` doc comment gives a worked example (`8/16/16/8 → 1/2/2/1`) that the code cannot actually produce (it always flattens toward `1/1/1/1`). Either the comment or the algorithm is wrong. - `js/data/landed-`

`cost.js` — unguarded division by `qty`; a receipt with `qty = 0` would write `Infinity / NaN` into a costing figure. Add a `qty > 0` guard.

5. `js/views/wholesaler.js` is a 68KB monolith holding 11 screens. It is navigable via its bottom route map and section banners (a dev can find any screen in under a minute), but it should be the next file split — natural seams: products+pricing+packs, inventory+intelligence, clients+team, settings.

6. **Minor:** two global event listeners (`sidenav.js` , `topbar.js`) are added on every shell mount without teardown — a small listener leak across login/logout cycles in a long-lived session. Low impact.

7. **Testing state is real but partial.** Only the **buyer** order path (a full-series pack) has been walked by a human end-to-end. Owner account-creation-of-wholesalers, the salesperson flow, prepack/ratio/open ordering, and cart editing have been *logic-verified but never walked* — the same condition that hid the packs bug. QA should walk each role deliberately. Also: the brand-new-owner *password/signup* path is unverified (existing owner logins have reused a prior Supabase Auth session).

8. **Security-header definition is triplicated** (`worker.js` , `_headers` , `index.html` meta-CSP) and kept in sync by hand — a policy change means editing three files or they drift. The CSP itself is strong (`script-src 'self'` , `object-src 'none'` , `frame-ancestors 'none'`).

10. Running and deploying

No build step. The front-end is static files; the app runs by serving the directory.

Deploy (current, automatic): push to `main` on GitHub → Cloudflare Workers Builds deploys automatically. There is no manual deploy command in normal operation.

Local run: serve the `wholesale-v2/` directory with any static file server (e.g. `npm run serve`) and open it. The app talks to the live Supabase project directly (the publishable key + project URL are in `js/lib/supabase-client.js`), so a local copy is fully functional against live data — useful for front-end work, but be aware you are hitting the live database.

Database changes: authored as new numbered migrations in `supabase/migrations/` and applied to the Supabase project (via the Supabase CLI/dashboard or an equivalent tool). The Cloudflare deploy does **not** run migrations — schema and app deploy are independent.

Edge functions: deployed to Supabase separately from the Cloudflare app (Supabase CLI `functions deploy` or dashboard).

11. Accounts and access (current state)

There are **no real customer accounts yet** — everything in the system is disposable test data. Before real wholesalers can be onboarded, their accounts must be created inside the app by the Owner.

- **Owner:** the Owner is the Supabase Auth user `oggi.teamz@gmail.com`. Sign in at the live URL on the **Owner / Wholesaler** tab with that email and its password. From the Owner Dashboard, the left menu (Wholesalers, Invites, Onboarding Queue) is where real wholesalers and their logins are created. These owner-side creation flows have not yet been walked by a human — expect to iterate.
- **Buyer login shape:** wholesaler code + username + password (e.g. the current test buyer is `mg` / `demo` / `demo1234` on the "Milano Garments" test wholesaler). Buyers are created by their wholesaler.
- **The owner bootstrap invite** used to activate the first owner has been consumed and referenced widely; rotate it once real owner access is settled.

12. One-paragraph summary for the CTO

OGGI Wholesale v2 is a live, multi-tenant B2B wholesale ordering app on Cloudflare Workers + Supabase, with a dedicated `wholesale_v2` Postgres schema (35 tables, 32 RPCs, RLS on every table), a no-build vanilla-JS front-end, and eight deployed edge functions. The code is genuinely well-documented and navigable — an independent audit of the full codebase confirmed consistent purpose-headers, comments that explain *why*, in-code security rationale, and comments that match behaviour, with only a couple of small documented exceptions. The core buyer order flow (including the hardest feature, the four selling models) is verified end-to-end on real data. Before public production it needs: the four unsaved migrations committed, the webhook/edge-function security posture hardened and signed off, and each remaining role walked by QA. It is ready to hand to testers now.